

题1② 火焰图生成与热点分析

—— 尖塔 vs 扁平形态对比与内核态函数分析
2026 CVM 校企合作考核 · 吴佳豪 · 2026-06

本报告使用 `perf record -F 99 -g` 采样 + FlameGraph 工具链，对计算密集型（`matrixprod`）与访存密集型（`rand-set`）两种负载生成 CPU 火焰图，对比“尖塔”与“扁平”形态差异，并解释内核态函数（`__do_page_fault`、`munmap→tlb_flush_mmu` 等）出现的原因及对性能的影响。火焰图原文见 `flamegraphs/*.svg`，本报告已将两张火焰图嵌入。

一、测试环境与工具链

项	值
CPU	鲲鹏 920 (TaiShan v110), aarch64, 4 核 @2.6GHz
采集条件	<code>taskset -c 0</code> 钉核, <code>perf record -F 99 -g</code> 采样, 30s
工具链	<code>perf 4.19.90</code> → <code>perf script</code> → FlameGraph (<code>stackcollapse-perf.pl</code> + <code>flamegraph.pl</code>)
栈回溯	<code>fp</code> (frame pointer), <code>stress-ng</code> 编译时保留 <code>fp</code> , 栈质量好

工具链命令（以 `matrixprod` 为例）：`taskset -c 0 perf record -F 99 -g -- stress-ng --cpu 1 --cpu-method matrixprod -t 30s`，再 `perf script > perf_matrixprod.data`，最后经 `stackcollapse-perf.pl` 与 `flamegraph.pl` 生成 SVG。`-F 99` 取 99Hz 采样以避免 100Hz 定时器共振，`-g` 抓取调用栈（`fp` 回溯）。

二、嵌入火焰图与热点函数标注

2.1 matrixprod 火焰图（尖塔）

函数	CPU 占比	所在层	说明
<code>__multf3</code>	57.35%	用户态 (libgcc 软浮点)	软浮点乘法 (<code>multiply float 3-arg</code>)
<code>__addtf3</code>	34.27%	用户态 (libgcc 软浮点)	软浮点加法
stress-ng 主循环	~6%	用户态	调度 stressor
其他	~3%	—	杂项

表 1 matrixprod 火焰图热点函数占比

热点结论：`__multf3` + `__addtf3` 合计约 92%——CPU 几乎全在软浮点运算库。单一热点集中，典型尖塔形态。

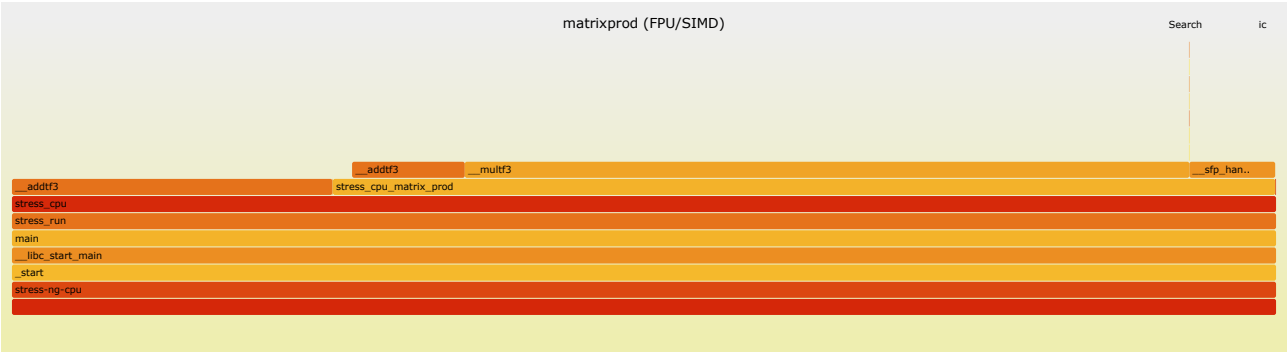


图 1 matrixprod 火焰图（尖塔形态，__multf3/__addtf3 软浮点库占 92%）

2.2 rand-set 火焰图（扁平）

区域	CPU 占比	代表函数	说明
用户态	~50%	stress_ng_method、__random_r、memset	stress-ng 随机写逻辑，分散多函数
内核态	~27%	munmap→tlb_flush_mmu、 pagemap_read→walk_page_range	内存管理（TLB 刷新、页表遍历）
其他	~23%	各种调度/系统调用	分散

表 2 rand-set 火焰图热点分布

热点结论：无单一函数占大头，时间分散在用户态多个函数 + 内核态多个函数，故呈扁平形态。

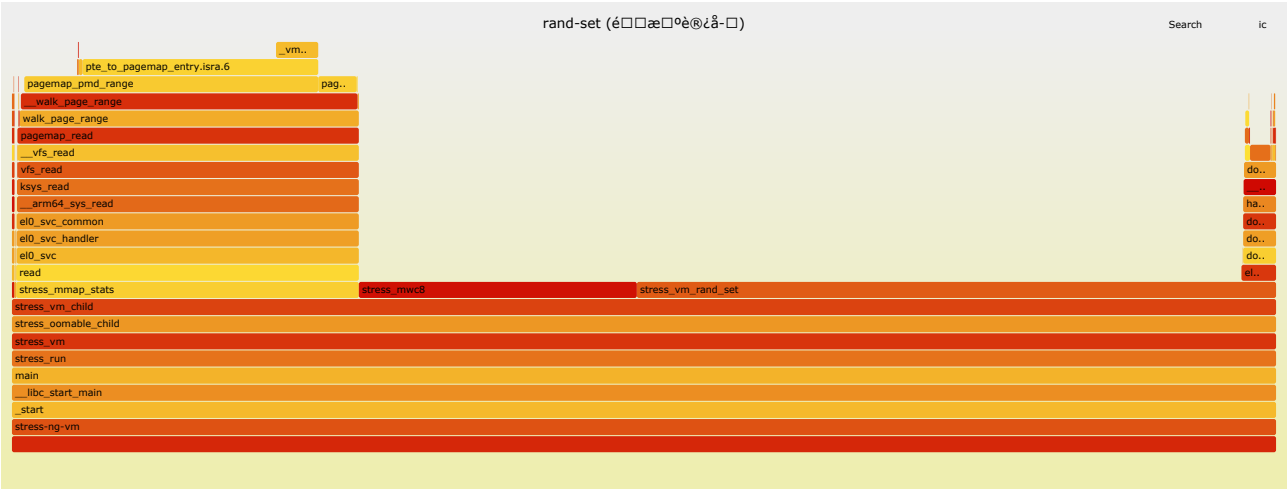


图 2 rand-set 火焰图（扁平形态，用户态 50% + 内核态 27%）

三、两种负载形态对比分析

3.1 宽度分布差异

维度	matrixprod	rand-set
顶层热点	__multf3 单函数 57%	无单函数超 20%
用户态占比	~99%（几乎纯用户态）	~50%
内核态占比	~1%	~27%
形态	**尖塔**（热点集中）	**扁平**（热点分散）

3.2 为什么尖塔 vs 扁平

matrixprod 尖塔的成因：计算密集型负载，CPU 几乎全在一个紧密循环里做浮点运算。
stress-ng 在 ARM 上用 libgcc 软浮点库（__multf3/__addtf3）——ARM 无硬件四精度浮点，浮点运算走软件库。软件库是一个紧凑函数，被采样到 92%+，故呈尖塔，且没有访存/系统调用分散 CPU 时间。

rand-set 扁平的成因：访存密集型负载，CPU 时间分散在“计算随机地址 + 写内存 + 等内存 + 内核内存管理”。用户态：随机数生成（__random_r）、内存写入（memset）等多个函数分担；内核态占比 27%：512M 随机写频繁触发 munmap→tlb_flush_mmu（TLB 刷新）和 pagemap_read→walk_page_range（页表遍历）。→ 没有单一热点，时间被多个函数瓜分，故呈扁平。

核心洞察：尖塔 = CPU-bound，CPU 全在一个计算函数里，优化该函数收益巨大；扁平 = memory-bound/branch-bound，CPU 时间分散在“等内存”和“内存管理”上，优化单个函数收益小。

四、内核态函数成因与性能影响

4.1 出现的内核态函数

rand-set 火焰图出现以下内核态函数（matrixprod 几乎没有）：

内核函数	出现原因	性能影响
munmap → tlb_flush_mmu	stress-ng 释放/重映射内存时调 munmap，触发 TLB 刷新	TLB 刷新后需重新填充，后续访问 TLB miss
pagemap_read → walk_page_range	内核遍历页表（procfs pagemap 接口）	多级页表遍历，多次内存访问
__do_page_fault（部分场景）	首次访问新分配页触发缺页	分配物理页 + 建立页表映射

4.2 为什么 rand-set 有内核态而 matrixprod 没有

对比	matrixprod	rand-set
内存操作	矩阵在栈/堆上，分配一次	512M 频繁随机写，触发内存管理
系统调用	几乎没有	munmap/pagemap 频繁
缺页	启动时少量	持续触发（访问新页）
→ 内核态占比	~1%	~27%

这解释了题1① 中 rand-set 的 sys time 30%：随机访存不仅压 cache/TLB，还压内核内存管理子系统。火焰图直观显示了“30% 的 CPU 时间花在 munmap/pagemap 等内核函数上”。

4.3 对性能的影响

- TLB 刷新（tlb_flush_mmu）：每次 munmap 后 TLB 被刷，后续访问要重新走页表 → TLB miss 升高（对应题1① rand-set 的 dTLB miss 0.166%）。
- 页表遍历（walk_page_range）：多级页表遍历是多次内存访问，本身也消耗 cache 带宽。

- 缺页处理 (`__do_page_fault`)：分配物理页 + 建映射有内核开销。
 - 内核态开销是访存型负载性能下降的重要组成，不能只看用户态。

五、结论

- 形态判断有效：matrixprod 尖塔（计算密集，`__multf3+__addtf3` 占 92%）vs rand-set 扁平（访存分散，用户态 50%+内核态 27%），符合“计算密集=尖塔、访存/分支=扁平”的理论预期。
- 热点函数精准定位：matrixprod 的软浮点库（libgcc `__multf3/__addtf3`）是 ARM 无硬件四精度浮点的体现，火焰图直接揭示了这一架构特性。
- 内核态函数解释了 sys time 差异：rand-set 的 `munmap→tlb_flush_mmu`、`pagemap_read→walk_page_range` 直观解释了题1① 的 sys 30%，且与 TLB miss 升高互为因果。
- 火焰图与 perf stat 互补：perf stat 量化“瓶颈多严重”（miss rate/sys 占比），火焰图定位“瓶颈在哪些函数”，两者配合才能完整分析。